Informe Técnico: Taller de Observabilidad con Stack Open Source

Práctica Final Integradora — Infraestructura TI, Universidad Andrés Bello



Leonardo Burgos

9 de noviembre de 2026

1. Paso a paso documentado

1.1 Entorno de trabajo

El stack se desplegó en GitHub Codespaces con Docker Compose v2. El repositorio es taller-observabilidad-final, rama main. Todos los servicios corren en la red observabilidad-net.



```
NAME                IMAGE                                          COMMAND                  SERVICE             CREATED             STATUS              PORTS
grafana             grafana/grafana:10.4.1                         "/run.sh"                grafana             About an hour ago   Up 13 minutes       0.0.0.0:3000->3000/
tcp, [::]:3000->3000/tcp
hotrod-microservices   jaegertracing/example-hotrod:1.56           "/go/bin/hotrod-linu…"   hotrod-app          About an hour ago   Up About an hour    0.0.0.0:8080->8080/
tcp, [::]:8080->8080/tcp, 8081-8083/tcp
jaeger              jaegertracing/all-in-one:1.56                  "/go/bin/all-in-one-…"   jaeger              About an hour ago   Up About an hour    4318/tcp, 5775/udp,
 5778/tcp, 9411/tcp, 14250/tcp, 0.0.0.0:4317->4317/tcp, [::]:4317->4317/tcp, 0.0.0.0:16686->16686/tcp, [::]:16686->16686/tcp, 14268/tcp, 6831-6832/udp
log-generator       alpine:latest                                  "/bin/sh -c 'while t…"   log-generator       About an hour ago   Up About an hour
loki                grafana/loki:2.9.6                             "/usr/bin/loki -conf…"   loki                About an hour ago   Up About an hour    0.0.0.0:3100->3100/
tcp, [::]:3100->3100/tcp
node-exporter       prom/node-exporter:v1.7.0                      "/bin/node_exporter"     node-exporter       About an hour ago   Up About an hour    0.0.0.0:9100->9100/
tcp, [::]:9100->9100/tcp
otel-collector      otel/opentelemetry-collector-contrib:0.97.0    "/otelcol-contrib --…"   otel-collector      About an hour ago   Up About an hour    4317/tcp, 55678-556
79/tcp, 0.0.0.0:4318->4318/tcp, [::]:4318->4318/tcp
prometheus          prom/prometheus:v2.51.0                        "/bin/prometheus --c…"   prometheus          About an hour ago   Up About an hour    0.0.0.0:9090->9090/
tcp, [::]:9090->9090/tcp
promtail            grafana/promtail:2.9.6                         "/usr/bin/promtail -…"   promtail            About an hour ago   Up About an hour
snmp-exporter       prom/snmp-exporter:v0.25.0                     "/bin/snmp_exporter …"   snmp-exporter       About an hour ago   Up About an hour    0.0.0.0:9116->9116/
tcp, [::]:9116->9116/tcp
snmp-simulator      polinux/snmpd:latest                           "/bootstrap.sh"          snmpd               About an hour ago   Up About an hour    161/tcp, 0.0.0.0:16
1->161/udp, [::]:161->161/udp
webhook-receiver    python:3.12-alpine                             "python /app/receive…"   webhook-receiver    43 minutes ago      Up 40 minutes
```

Figura 1. Servicios del stack levantados con Docker Compose (12 contenedores en estado Up).

1.2 Preparación y despliegue

Antes de levantar el entorno se creó el archivo .env a partir de .env.example, definiendo la contraseña de administrador de Grafana. El archivo no se versiona porque está en .gitignore. Luego se validó la sintaxis con docker compose config y se levantó el entorno:

```
docker compose config > /dev/null && echo "compose OK"
docker compose up -d
docker compose ps
```

1.3 Validación de componentes

Grafana se conectó a dos fuentes de datos: Prometheus (http://prometheus:9090) para las métricas y Loki (http://loki:3100) para los logs. Ambas se configuran desde grafana/provisioning/datasources/datasources.yml, así que quedan disponibles al levantar el contenedor sin pasos manuales.

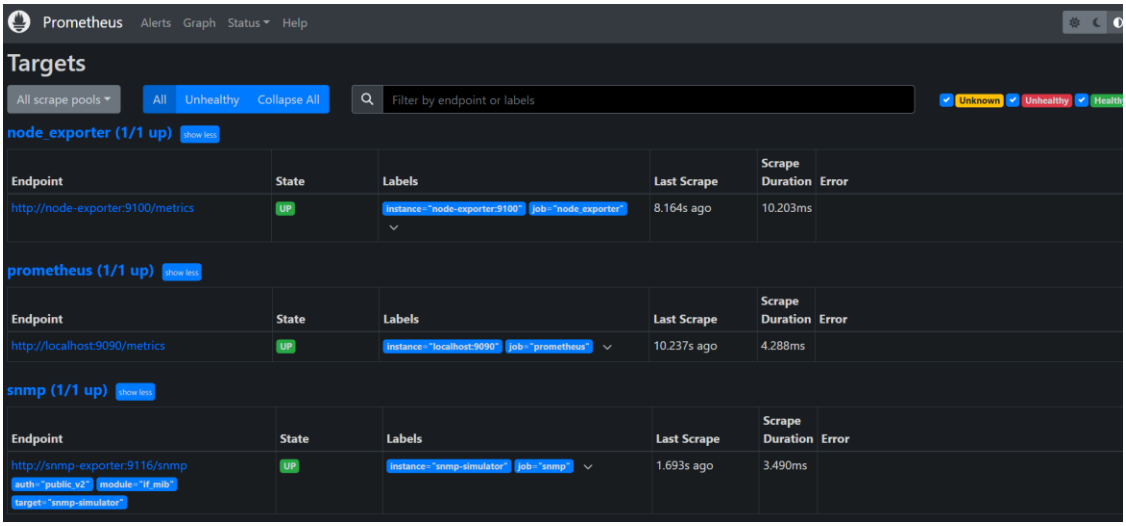


Figura 2. Targets de Prometheus: node_exporter, prometheus y snmp en estado UP.

Para los logs, se verificó que Loki recibe las etiquetas de severidad y de servicio:

curl -s localhost:3100/loki/api/v1/labels

La respuesta incluye env, filename, job, level y service.

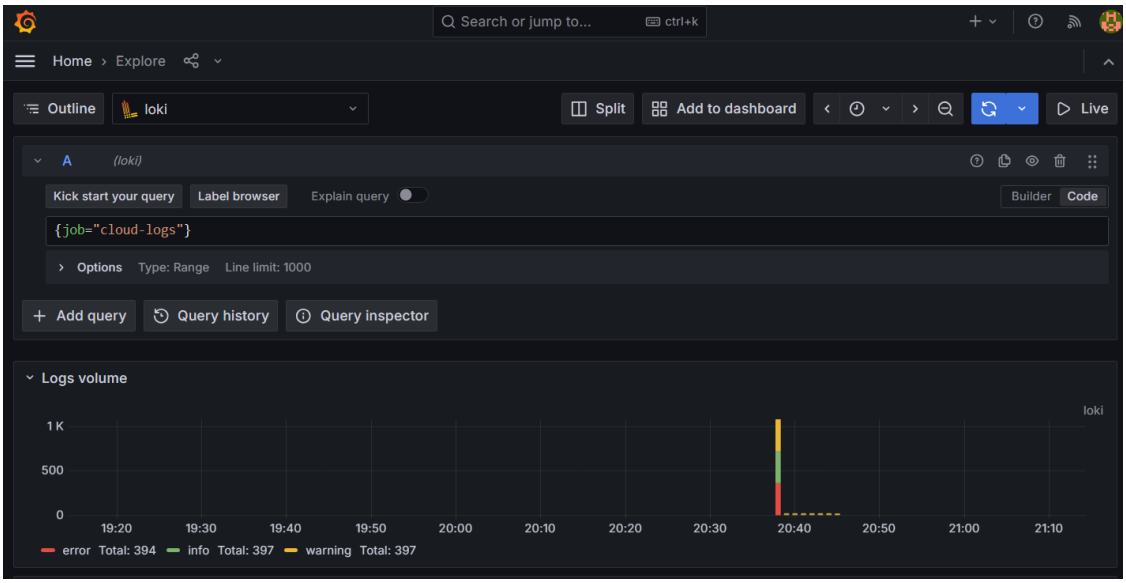


Figura 3. Explore de Loki con datasource Loki y la consulta {job="cloud-logs"}.

1.4 Problemas encontrados y solución

Durante el despliegue aparecieron cuatro problemas. Cada uno se resolvió así:

Problema	Causa	Solución aplicada
node-exporter no arrancaba: *path / is mounted on / but it*	La opción rslave exige un origen *shared*, y el entorno	Se usó /:/host:ro, de solo lectura, sin rslave

Problema	Causa	Solución aplicada
is not a shared or slave mount	de Codespaces no lo cumple	
Promtail no arrancaba: *did not find expected key* (línea 27)	La etapa timestamp tenía una indentación incorrecta	Se corrigió la indentación del bloque pipeline_stages
La regla de CPU no se evaluaba: *reducer mode 'strict' is not supported*	La expresión de reducción usaba un modo no válido	Se eliminó la línea settings de la expresión B
El receptor del webhook no mostraba los payloads	La versión del archivo remoto respondía 200 sin imprimir nada	Se corrigió receiver.py para imprimir cada payload con flush=True

2. Justificación técnica

2.1 Consultas PromQL

Uso de CPU. Se calcula la tasa de segundos en modo idle de cada núcleo, se promedia por instancia y se resta de 1. Multiplicado por 100, da el porcentaje de tiempo no ocioso:

```
100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

Se usa avg y no sum porque el resultado debe ser un porcentaje independiente del número de núcleos.

Ancho de banda en Mbps. ifInOctets e ifOutOctets son contadores en bytes. La tasa de 5 minutos se multiplica por 8 para pasar a bits y se divide entre 10^6 para expresarla en Mbps:

```
rate(ifInOctets{ifDescr="eth0"}[5m]) * 8 / 1000000
```

```
rate(ifOutOctets{ifDescr="eth0"}[5m]) * 8 / 1000000
```

Grafana muestra una advertencia informativa porque ifOutOctets no termina en _total, _sum, _count o _bucket. Es un contador monótono exportado por snmp_exporter, así que rate() es válido. No se renombró la métrica para conservar el nombre de la MIB IF-MIB.

Almacenamiento. El node-exporter monta el disco del host en /host en modo solo lectura. En el entorno de Codespaces, el gauge usa el punto de montaje /etc/hostname, que pertenece al mismo dispositivo (/dev/loop4, ext4):

```
100 * (1 - node_filesystem_avail_bytes{mountpoint="/etc/hostname",fstype!~"tmpfs|overlay"} / node_filesystem_size_bytes{mountpoint="/etc/hostname",fstype!~"tmpfs|overlay"})
```

En una máquina Linux con el montaje original (rslave), el mismo gauge puede usar mountpoint="/".

2.2 Pipeline de Promtail (Loki)

1. **`json`:** extrae `level`, `service`, `msg` y `timestamp` de cada línea.
2. **`timestamp`:** usa el timestamp del log como hora del evento, en lugar de la hora de ingesta.
3. **`labels`:** promueve `level` y `service` a etiquetas indexadas. Son de baja cardinalidad (tres niveles y pocos servicios), por eso son adecuadas como etiquetas. `msg` no se promueve porque sus valores casi no se repiten y generaría demasiadas series.
4. **`output`:** deja `msg` como contenido de la línea almacenada en Loki. El nivel y el servicio siguen disponibles como etiquetas, así que no se pierde información útil para filtrar.

Ejemplo de una línea de log de entrada:{

```
  "level": "ERROR",
  "service": "db-connector",
  "msg": "Timeout al conectar con replica secundaria"
}
```

Filtro LogQL para la severidad de error:

logql

```
{job="cloud-logs", level="ERROR"}
```

Este filtro usa `level` como etiqueta indexada, así que Loki busca directamente en los flujos de error sin leer el contenido de cada línea. Por eso se eligió `level` como etiqueta y no como texto dentro del mensaje.

2.3 Enrutamiento de trazas

HotROD envía trazas por OTLP a otel-collector:4318. El collector aplica un procesador batch y reenvía por OTLP/gRPC a jaeger:4317. Jaeger almacena las trazas y las muestra en el puerto 16686.

2.4 Regla de alerta y notificación

- **Regla:** *CPU alta*, en la carpeta *Operaciones*, grupo cpu-alerts. Evalúa el uso de CPU con la expresión de la sección 2.1, reduce el resultado con last y compara contra el umbral de **80 %**. La condición debe mantenerse **2 minutos** (for: 2m) antes de pasar a *Firing*.
- **Contact point:** webhook-simulado, que envía las notificaciones por POST a http://webhook-receiver:8000.
- **Política de notificaciones:** la alerta se enruta a webhook-simulado con la configuración por defecto de Grafana.

2.5 Decisiones de entorno

El entorno de Codespaces no permite el montaje rslave. Por eso se usó /:/host:ro para node-exporter y se ajustó el punto de montaje del gauge. Esta es una limitación del entorno, no del diseño: en una máquina Linux propia, la configuración original con rslave es equivalente.

3. Evidencia de funcionamiento

3.1 Dashboard operativo

El dashboard *Dashboard Operativo* reúne en una sola vista cuatro paneles: el tráfico de red de eth0 en Mbps, para las direcciones entrante y saliente; el uso de almacenamiento del disco del host; el uso de CPU por instancia, y los logs en tiempo real desde Loki, con su nivel de severidad como etiqueta.

El tráfico aparece en 0.000 Mbps porque el simulador SNMP genera muy poco tráfico. Es un valor real y no un error de consulta.



Figura 4. Dashboard operativo con tráfico de red eth0 en Mbps (entrante y saliente), uso de almacenamiento, uso de CPU por instancia y logs en tiempo real desde Loki.

3.2 Regla de alerta y webhook

La regla *CPU alta* evalúa el uso de CPU con un umbral de 80 % durante 2 minutos. Al superar el umbral, pasa a estado *Firing* y Grafana envía el payload al contact point webhook-simulado, que apunta a http://webhook-receiver:8000.

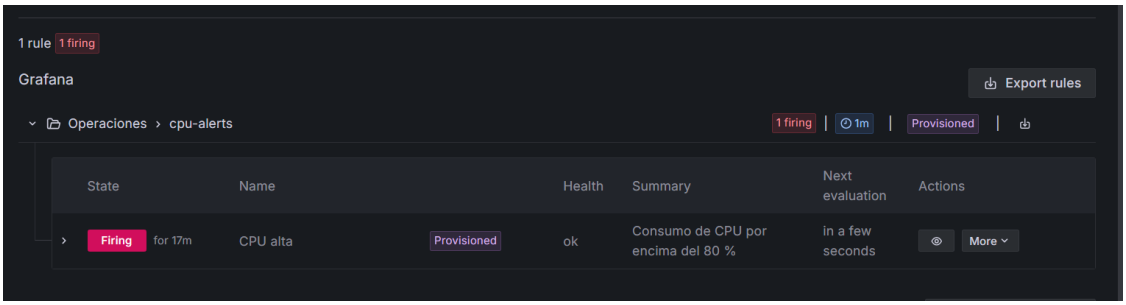


Figura 5. Regla “CPU alta” en estado Firing tras superar el umbral de 80 % de uso de CPU (node-exporter:9100).

El payload de la notificación de *Firing*, enviado a las **01:14:50 UTC**, registra un uso de CPU de **99.99 %** (valor B) con la condición de umbral cumplida (valor C = 1):


```
"receiver": "webhook-simulado",
"status": "firing",
"alerts": [
  {
    "status": "firing",
    "labels": {
      "alertname": "CPU alta",
      "grafana_folder": "Operaciones",
      "instance": "node-exporter:9100",
      "severity": "warning"
    },
    "annotations": {
      "summary": "Consumo de CPU por encima del 80 %"
    },
    "startsAt": "2026-11-07T01:14:50Z",
    "endsAt": "0001-01-01T00:00:00Z",
    "generatorURL": "http://localhost:3000/alerting/grafana/cpu-alta/view?orgId=1",
    "fingerprint": "716ff858f881885e",
    "silenceURL": "http://localhost:3000/alerting/silence/new?alertmanager=grafana&matcher=alertname%3DCPU+alta&matcher=grafana_folder
%3DOperaciones&matcher=instance%3Dnode-exporter%3A9100&matcher=severity%3Dwarning&orgId=1",
    "dashboardURL": "",
    "panelURL": "",
    "values": {
      "B": 99.99122807017544,
      "C": 1
```


Figura 6. Payload enviado al webhook cuando “CPU alta” pasa a Firing (01:14:50 UTC). B = 99.99 %, C = 1.

Como respaldo, el ciclo anterior se resolvió a las **00:58:50 UTC**, con valores B = 68.37 % y C = 0, lo que confirma la transición de *Firing* a *Normal*.

Los payloads de TestAlert, generados con el botón *Test* del contact point, sirven únicamente para comprobar la conectividad entre Grafana y el receptor. No corresponden a la regla de CPU.

3.3 Trazabilidad distribuida (APM)

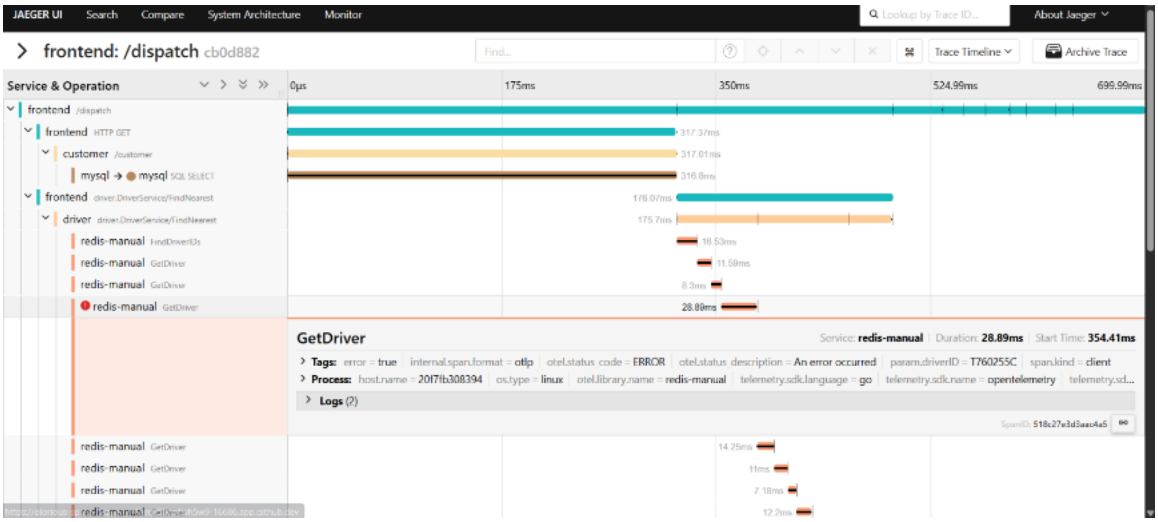


Figura 7. Traza de frontend /dispatch en Jaeger; el span mysql SELECT del servicio customer dura unos 317 ms y es el cuello de botella observado.

En la traza de frontend /dispatch, de unos 700 ms en total, el span mysql SELECT del servicio customer dura unos 317 ms, cerca del 45 % del tiempo. Este es el cuello de botella observado: la latencia está en la consulta a la base de datos y no en la lógica de la aplicación.Además, el span redis-manual GetDriver registra un error (otel.status_code=ERROR), con una duración de unos 29 ms. Es una observación secundaria, porque el error no domina el tiempo de la transacción.

4. Conclusiones y lecciones aprendidas

La implementación de este stack open source (Prometheus, Loki, Grafana, Jaeger y OpenTelemetry) demostró ser una solución robusta para los problemas de observabilidad abordados en el taller. Centralizar métricas de hardware y de red, unificar logs estructurados y visualizar trazas de microservicios en un mismo ecosistema permite al SRE detectar incidentes silenciosos antes de que afecten a la plataforma.

Las lecciones más útiles fueron operativas. Los errores más costosos fueron de formato (indentación en YAML, un modo de reducción no válido) y de entorno (montajes de host). Esto refuerza que la configuración debe validarse antes de desplegar, y que los objetos de Grafana conviene provisionarlos desde archivos, porque así sobreviven a la recreación de los contenedores.

También quedó claro que una alerta sin evidencia no sirve. Hubo que distinguir entre una prueba de conectividad (TestAlert) y la notificación real de la regla, y documentar la hora y los valores de cada payload. Por último, la trazabilidad distribuida fue lo que permitió identificar la latencia dentro del código, algo que las métricas de hardware por sí solas no mostraban.